Jeremy Barton
Term Project: Milestone 1
DSC550-T302
Spring 2026

WigglNET: Data Mining and Brokerage Service for E-Commerce Niche Market Sellers and Small to Mid-size Product Manufacturers

Business Question

Small e-commerce sellers and product manufacturers struggle to understand what is popular, why and what will be popular in the future. This data could be retrieved by a search engine or AI, but there is more value in using advanced models specially trained to identify people, organizations or entities related to a specific product category to discover these insights rather than a Large Language Model trained on everything, that happens to have picked up data related to a particular target market.

The question being asked in this business problem is *"Can we predict the sentiment of keywords and entities related to a certain product niche based on a classification model trained on hundreds of web articles related to that category within a certain time frame?*"

A great way to examine this is to target the `sentiment` column of ARTICLE_ENTITY_SENTIMENT table, which is the variable tied to the vibe of a keyword or entity. This table is the product of a pipeline that scrapes, reads and sets the sentiment of the related blog article keywords and enities. It is broken down into three classes: **positive, neutral and negative**. This is one of the most useful variables to analyze becuase it tells a client how to make decisions about these entities.

Exploring the Data

MariaDB is the database system used for development purposes.

In [34]:
```
# Imports and Processing
import sys
sys.path.insert(0, '/home/jeremy/Documents/GitHub/wigglnet')

import mariadb
import pandas as pd
from app.shared_utils.db_utils import get_db_config

# Get database configuration from .env file
config = get_db_config()

# Connect to DB
try:
    conn = mariadb.connect(
        host=config['host'],
        user=config['user'],
        password=config['password'],
        database=config['database'],
        port=config['port']
    )
    print("Successfully connected to MariaDB database")
except mariadb.Error as e:
    print(f"Error connecting to MariaDB Platform: {e}")
    sys.exit(1)

# Get the cursor
cur = conn.cursor()
```

```
Successfully connected to MariaDB database
```

The SQL query is pulling back combined article, keyword, entity and sentiment information by joining to four other tables on either ARTICLE_ID or KEYWORD_ID. Using multiple tables with a JOIN, an overview of the joined insight data is stored in a dataframe. Because this is time series data, the query can ignore all rows where PUBLISHED_DATE is NULL.

The query includes the CONFIDENCE field from ARTICLE_ENTITY_SENTIMENT, which indicates the RoBERTa model's confidence level (0-1) in the sentiment classification for each entity.

In [35]:
```sql
# SQL Query to retrieve article data with keywords, entities, and sentiment
query = """
    SELECT
        a.ARTICLE_ID,
        a.TITLE,
        a.PUBLISHED_DATE,
        k.KEYWORD_TEXT,
        e.ENTITY_TEXT,
        e.ENTITY_TYPE,
        aes.SENTIMENT,
        aes.CONFIDENCE,
        aes.MENTIONS
    FROM WEBDATA.ARTICLES a
    LEFT JOIN WEBDATA.ARTICLE_KEYWORDS ak ON a.ARTICLE_ID = ak.ARTICLE_ID
    LEFT JOIN WEBDATA.KEYWORDS k ON ak.KEYWORD_ID = k.KEYWORD_ID
    LEFT JOIN WEBDATA.ARTICLE_ENTITY_SENTIMENT aes ON a.ARTICLE_ID = aes.ARTICLE_ID
    LEFT JOIN WEBDATA.ENTITIES e ON aes.ENTITY_ID = e.ENTITY_ID
    WHERE PUBLISHED_DATE IS NOT NULL
     -- AND published_date BETWEEN CURRENT_DATE - INTERVAL 1 YEAR AND CURRENT_DATE;
"""

# Execute query and load results into a DataFrame
df = pd.read_sql(query, conn)
```

```
/tmp/ipykernel_8994/3352011010.py:23: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection)
or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQ
LAlchemy.
  df = pd.read_sql(query, conn)
```

In [36]:
```
# Display basic info about the dataset
print(f"Dataset shape: {df.shape}")
print(f"\nColumn names: {df.columns.tolist()}")
```

```
Dataset shape: (19367, 9)

Column names: ['ARTICLE_ID', 'TITLE', 'PUBLISHED_DATE', 'KEYWORD_TEXT', 'ENTITY_TEXT', 'ENTITY_TYPE', 'SENTIMENT',
'CONFIDENCE', 'MENTIONS']
```

How many articles are in this dataset?

In [37]:
```
total_articles = df["ARTICLE_ID"].nunique()
print(f"There are {total_articles} articles in this dataset.")
```

```
There are 132 articles in this dataset.
```

If there are only 132 articles, why are there 19,637 rows?

This dataset includes information on metadata about keywords and enitities in an article, not just a list of each article. Every keyword has an entity text, type, sentiment and number of mentions.

What are the max and min published dates?

In [38]:
```
max_pub = df["PUBLISHED_DATE"].max()
print(f"Max published date is {max_pub}")

min_pub = df["PUBLISHED_DATE"].min()
print(f"Min published date is {min_pub}")
```

```
Max published date is 2026-04-10 00:00:00
Min published date is 2022-04-01 00:00:00
```

Keyword and Enitiy Sentiment Analysis by Mentions

These visualizations analyze the positive, neutral or negative sentiment of entities and keywords from hundreds of blog articles related to Home Decor/Furniture. These can indicate the direction a seller or manufacturer should go terms of marketing or collaborations for networking. This could also help guide a business in determining which colors and/or materials are "in" or "out" of style based on the time range of the analysis.

Stacked Bar Chart of Entity Sentiment by Top 25 Keywords and Entities Over the Past Year

A stacked bar chart will show how sentiment is distributed across the most frequently mentioned keywords. The colors within the bar represent the positive, neutral and negative sentiments. `sentiment` for a keyword or entity can have three options: positive, neutral and negative.

Data Preparation

In [39]:
```
# Import visualization libraries
import matplotlib.pyplot as plt
import numpy as np

# Set visualization style
plt.style.use('ggplot')
```

Aggregate Data by Keyword and Sentiment

Sum up the mentions for each keyword-sentiment combination, then identify the top 10 keywords sorted by total mentions.

In [40]:
```
# aggregate data by keyword and sentiment
keyword_sentiment = df.groupby(['KEYWORD_TEXT', 'SENTIMENT'])['MENTIONS'].sum().reset_index()
                                                                    # summing the mentions
# using nlargest() to find the top 10 keywords
top_keywords = df.groupby('KEYWORD_TEXT')['MENTIONS'].sum().nlargest(25).index.tolist()

# filter to only include top keywords
keyword_sentiment_top = keyword_sentiment[keyword_sentiment['KEYWORD_TEXT'].isin(top_keywords)]
```

In [41]:
```
# print out top 10 keywords
print(f"Top 10 keywords: {top_keywords}")
print(f"\nAggregated data shape: {keyword_sentiment_top.shape}")
```

```
Top 10 keywords: ['home', 'space', 'room', 'design', 'designer', 'color', 'paint', 'house', 'colors', 'ideas', 'ki
tchen', 'designers', 'feel', 'look', 'wall', 'walls', 'bedroom', 'living', 'wood', 'white', 'everything', 'green',
'beautiful', 'work', 'small']

Aggregated data shape: (66, 3)
```

Right now the sentiment data is in its vertical form, meaning each of the three options are in rows under the `sentiment` column, and there are likely duplicates. Pivoting this data reshapes it so each sentiment becomes a column. This makes creating stacked bar charts easier.

Pivoting the Data for Stacking

In [42]:
```
# pivot the data so sentiments become columns
pivot_data = keyword_sentiment_top.pivot(index='KEYWORD_TEXT',
                                         columns='SENTIMENT',
                                         values='MENTIONS').fillna(0)

# reorder columns - bottom to top = negative to postitve
sentiment_order = ['negative', 'neutral', 'positive']
pivot_data = pivot_data.reindex(columns=sentiment_order, fill_value=0)
                        # .reindex() is used to
# sort by total mentions (descending)
pivot_data['total'] = pivot_data.sum(axis=1)
pivot_data = pivot_data.sort_values('total', ascending=False).drop('total', axis=1)
```

In this pivioted data, each sentiment option is its own column to identify a sentiment score for each keyword.

Visualization

Creating the Stacked Bar Chart

Each bar represents a keyword, with segments showing the distribution of positive, neutral, and negative sentiment.

In [43]:
```
# Step 3: Create the stacked bar chart
fig, ax = plt.subplots(figsize=(12, 6))

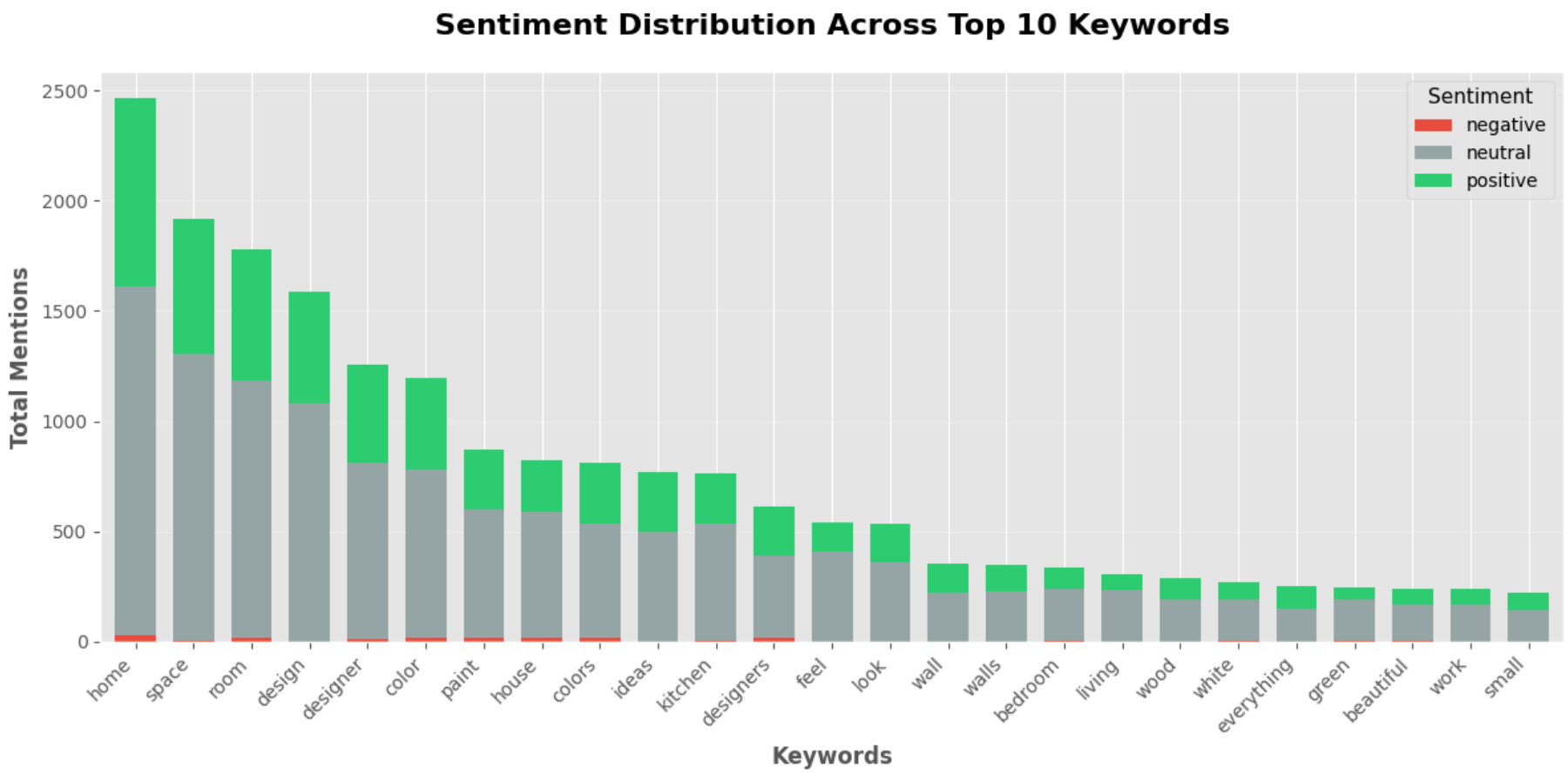
# Define colors for each sentiment
colors = {
    'negative': '#e74c3c',   # Red
    'neutral': '#95a5a6',    # Gray
    'positive': '#2ecc71'    # Green
}

# Create the stacked bar chart
pivot_data.plot(kind='bar',
                stacked=True,
                ax=ax,
                color=[colors[col] for col in pivot_data.columns],
                width=0.7)

# Customize the chart
ax.set_title('Sentiment Distribution Across Top 10 Keywords', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Keywords', fontsize=12, fontweight='bold')
ax.set_ylabel('Total Mentions', fontsize=12, fontweight='bold')
ax.legend(title='Sentiment', title_fontsize=11, fontsize=10)
ax.set_xticklabels(pivot_data.index, rotation=45, ha='right')

# Add grid for better readability
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```



Observation

Total mentions for a keyword are represented by the height of each bar. The segment colors (green = positive, gray = neutral, red = negative) are the proportions of each sentiment category. Keywords with more green indicate positive associations, while those with more red suggest negative coverage. However, keywords dominated in gray are mentioned in neutral contexts.

In practice, a client might use this chart to determine what topics to write most about on social media or in a decision for an upcoming product order.

Visualizing Entity Sentiment by Overall Total Mentions

This approach examines sentiment distribution across the most frequently mentioned keywords. The words most used amongst all blog website articles are shown with thehir positive and negative sentiment ratios within a bar chart.

Data Preparation

Both visualizations reuse the same base aggregation (`entity_sentiment`) to avoid duplication. The differences are:

1. **First chart: Top 15 entities by total mention volume**
2. **Second chart: Top 10 entities ordered by negative sentiment (below)**

In [44]:
```
# Aggregate data by entity and sentiment (reused across all entity visualizations)
entity_sentiment = df.groupby(['ENTITY_TEXT', 'SENTIMENT'])['MENTIONS'].sum().reset_index()

# Visualization 1: Top 15 entities by total mentions
top_entities_by_volume = df.groupby('ENTITY_TEXT')['MENTIONS'].sum().nlargest(25).index.tolist()
entity_sentiment_top_volume = entity_sentiment[entity_sentiment['ENTITY_TEXT'].isin(top_entities_by_volume)]

print(f"Top 15 entities by total mentions: {top_entities_by_volume}")
print(f"\nAggregated data shape: {entity_sentiment_top_volume.shape}")
```

```
Top 15 entities by total mentions: ['House', 'Benjamin Moore', 'ED', 'ER', 'Elle Decor', 'Ring', 'THE ENTIRE HOUSE
', 'ELLE', 'Limewash', 'Sherwin-Williams', 'Crown', 'Brown', 'Whole Home', 'Gaines', 'New York', 'Brooklyn', 'Farr
ow & Ball', 'Satin', 'French', 'Meghan Shouse', 'Roman', 'The Home Depot', 'Hayslett', 'Instagram', 'AV']

Aggregated data shape: (39, 3)
```

Pivoting the data for stacking

In [45]:
```
# Pivot the data so sentiments become columns
pivot_entity_volume = entity_sentiment_top_volume.pivot(index='ENTITY_TEXT',
                                                        columns='SENTIMENT',
                                                        values='MENTIONS').fillna(0)

# Reorder columns to control the stack order
pivot_entity_volume = pivot_entity_volume.reindex(columns=sentiment_order, fill_value=0)

# Sort by total mentions (descending)
pivot_entity_volume['total'] = pivot_entity_volume.sum(axis=1)
pivot_entity_volume = pivot_entity_volume.sort_values('total', ascending=False).drop('total', axis=1)
```

Visualization

Creating the stacked bar chart for entities

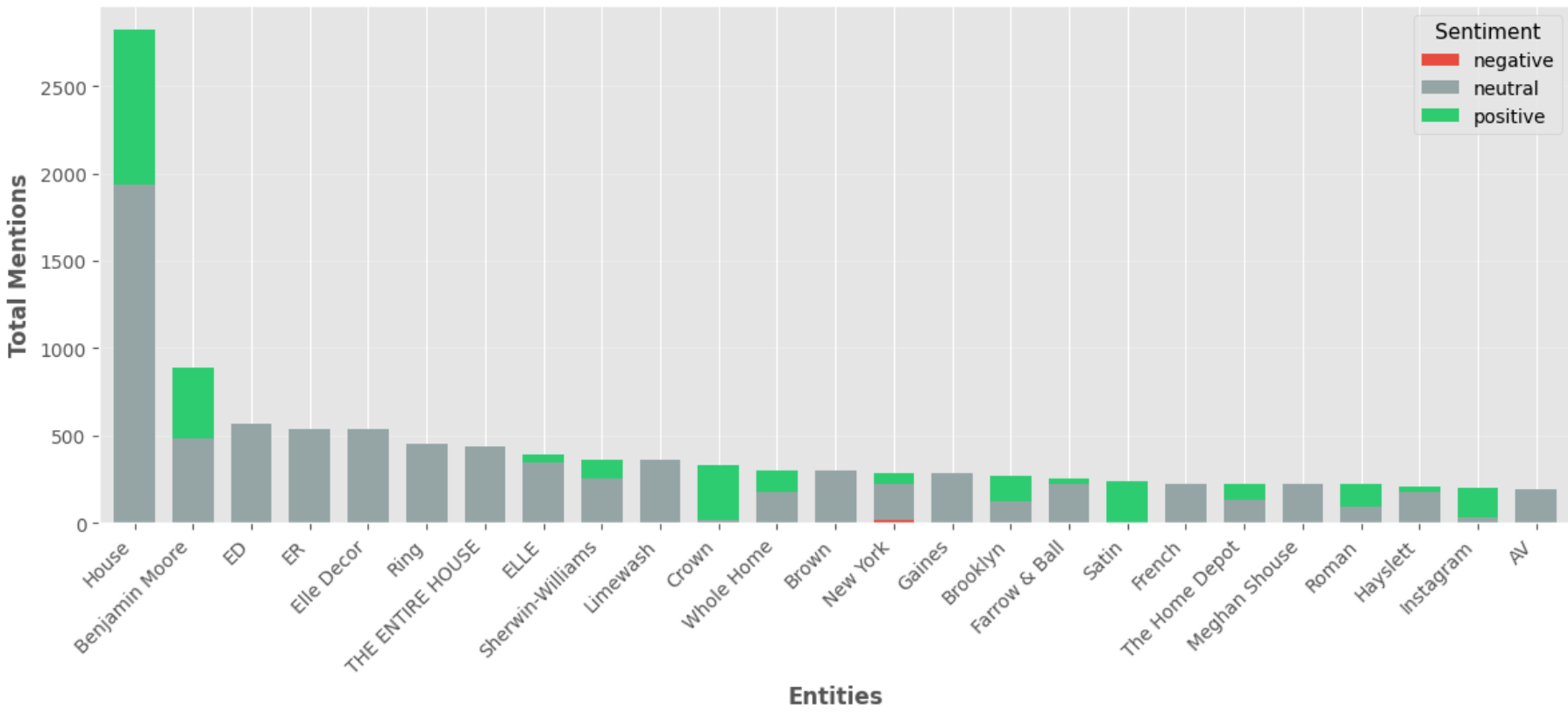
In [46]:
```
# Reusable visualization function parameters
fig, ax = plt.subplots(figsize=(12, 6))

pivot_entity_volume.plot(kind='bar',
                         stacked=True,
                         ax=ax,
                         color=[colors[col] for col in pivot_entity_volume.columns],
                         width=0.7)

# Customize the chart
ax.set_title('Sentiment Distribution Across Top 15 Entities', fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Entities', fontsize=12, fontweight='bold')
ax.set_ylabel('Total Mentions', fontsize=12, fontweight='bold')
ax.legend(title='Sentiment', title_fontsize=11, fontsize=10)
ax.set_xticklabels(pivot_entity_volume.index, rotation=45, ha='right')

# Add grid for better readability
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```



Observation

This chart shows sentiment distribution for named entities such as people, organizations, brands and locations, mentioned in the blog articles. It can be used to understand what entities drive certain topics.

However, a clear observation about this chart is that there is no "negative" sentiment accross any of the top 10 entities. (Except for New York, which is an outlier)

There could be a couple reasons behind this, but the most obvious one is that the most talked about keywords would never include negative sentiment, or else it wouldn't have shown up on the top 10 in the first place.

Instead of pulling the top 10 directly, for entities, we should include a condition to pull in the top 10 entities with a *with a negative or positive sentiment* attached to it and order by NEGATIVE sentiment.

This way, we can see what top entities have the highest negative sentiment and discover the most mentioned entity with both postitive and negative sentiment.

Another analysis...

Identifying Entities with Positive or Negative Sentiment, Ordered by Negative Sentiment

This approach focuses on entities with strong sentiment signals (positive or negative), excluding purely neutral entities, and orders them by negative sentiment to highlight problematic entities first. A client might use this graph to understand what brands or keywords to avoid in future business decisions.

Data Preparation

In [47]:
```
entity_sentiment = df.groupby(['ENTITY_TEXT', 'SENTIMENT'])['MENTIONS'].sum().reset_index()

# find entities that have positive OR negative sentiment (not just neutral)
entity_sentiment_set = df.groupby('ENTITY_TEXT')['SENTIMENT'].apply(lambda x: set(x))
                                                              # convert each groups sentiment values to a Python set

# filter to entities that have 'positive' or 'negative' in their sentiment set
entities_with_sentiment = entity_sentiment_set[
    entity_sentiment_set.apply(lambda x: 'positive' in x or 'negative' in x)
].index.tolist() # convert to list

print(f"Found {len(entities_with_sentiment)} entities with positive or negative sentiment")
```

```
Found 399 entities with positive or negative sentiment
```

In the previous code, the data was grouped by `ENTITY_TEXT` and `SENTIMENT` and `.apply(lamdba x: set(x))` converts the sentiment valus to a Python set.

The below cell creates a `netagtive_mentions` variable which will be used to count the mentions for the new graph.

In [48]:
```
# Get negative mention counts for these entities
negative_mentions = df[
    (df['ENTITY_TEXT'].isin(entities_with_sentiment)) &
    (df['SENTIMENT'] == 'negative')
].groupby('ENTITY_TEXT')['MENTIONS'].sum()
```

Then, most important for this graph, entires with negative OR positive sentiment exists are saved into a variable.

In [49]:
```
# Find entities that have positive OR negative sentiment (not just neutral)
entity_sentiment_set = df.groupby('ENTITY_TEXT')['SENTIMENT'].apply(lambda x: set(x.unique()))

# Filter to entities that have 'positive' or 'negative' in their sentiment set
entities_with_sentiment = entity_sentiment_set[
    entity_sentiment_set.apply(lambda x: 'positive' in x or 'negative' in x)
].index.tolist()

print(f"Found {len(entities_with_sentiment)} entities with positive or negative sentiment")
```

```
Found 399 entities with positive or negative sentiment
```

Now sorting by negative sentiment in decending order should show the top 10 negatively mentioned entities.

In [50]:
```
# sort by negative mentions (descending) and get top 10
all_sentiment_entities = pd.Series(0, index=entities_with_sentiment)
all_sentiment_entities.update(negative_mentions)
top_entities_by_negative = all_sentiment_entities.nlargest(25).index.tolist()

# filter to only include selected entities
entity_sentiment_top_negative = entity_sentiment[entity_sentiment['ENTITY_TEXT'].isin(top_entities_by_negative)]

print(f"\nTop 10 entities (ordered by negative sentiment):")
for i, entity in enumerate(top_entities_by_negative, 1):
    neg_count = all_sentiment_entities[entity]
    print(f"  {i}. {entity}: {int(neg_count)} negative mentions")

print(f"\nAggregated data shape: {entity_sentiment_top_negative.shape}")
print(f"\nSentiment breakdown for selected entities:")
```

```
Top 10 entities (ordered by negative sentiment):
  1. Boozer: 60 negative mentions
  2. Kohler: 45 negative mentions
  3. Scott: 45 negative mentions
  4. Kikuchi: 30 negative mentions
  5. Sheboygan: 30 negative mentions
  6. Song: 30 negative mentions
  7. Dafna Kikuchi: 15 negative mentions
  8. Hamptons: 15 negative mentions
  9. Joanna Saltz: 15 negative mentions
  10. Lindsey Herod: 15 negative mentions
  11. Nate Berkus: 15 negative mentions
  12. New York: 15 negative mentions
  13. Rayman Boozer: 15 negative mentions
  14. Sasha Bikoff: 15 negative mentions
  15. Victorian: 15 negative mentions
  16. dictionary.com: 15 negative mentions
  17. iPhone: 15 negative mentions
  18. ADA: 0 negative mentions
  19. Agárdy: 0 negative mentions
  20. Akbary: 0 negative mentions
  21. Alexander Design: 0 negative mentions
  22. Alexander Reid: 0 negative mentions
  23. Alexandra Gater: 0 negative mentions
  24. Alexandra Kaehler: 0 negative mentions
  25. Alexandra Loew: 0 negative mentions

Aggregated data shape: (40, 3)

Sentiment breakdown for selected entities:
```

Pivot and visualize the data (ordered by negative sentiment):

In [51]:
```
# Pivot the data so sentiments become columns (same logic as first viz)
pivot_entity_negative = entity_sentiment_top_negative.pivot(index='ENTITY_TEXT',
                                                            columns='SENTIMENT',
                                                            values='MENTIONS').fillna(0)

# Reorder columns to control the stack order
pivot_entity_negative = pivot_entity_negative.reindex(columns=sentiment_order, fill_value=0)

# Sort by NEGATIVE mentions (descending) - key difference from first visualization
pivot_entity_negative = pivot_entity_negative.sort_values('negative', ascending=False)
```

Visualization

In the following graph, because of the new ordering, entities with the highest number of negative mentions should be identified first, and then entities with positive OR negative mentions.

In [52]:
```
# create stacked bar chart (same visualization logic as first chart)
fig, ax = plt.subplots(figsize=(12, 6))

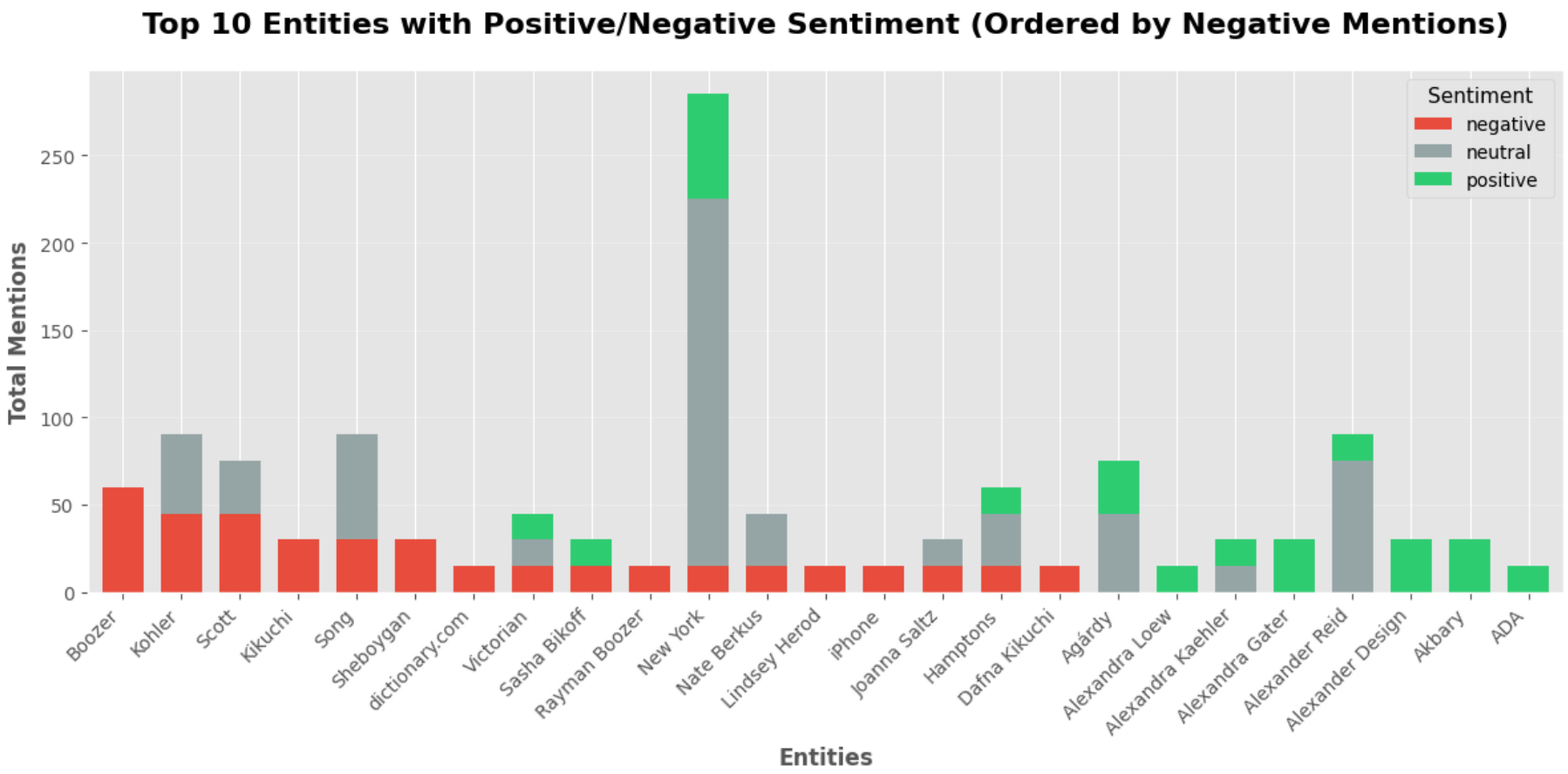
pivot_entity_negative.plot(kind='bar',
                           stacked=True,
                           ax=ax,
                           color=[colors[col] for col in pivot_entity_negative.columns],
                           width=0.7)

# Customize the chart
ax.set_title('Top 10 Entities with Positive/Negative Sentiment (Ordered by Negative Mentions)',
             fontsize=16, fontweight='bold', pad=20)
ax.set_xlabel('Entities', fontsize=12, fontweight='bold')
ax.set_ylabel('Total Mentions', fontsize=12, fontweight='bold')
ax.legend(title='Sentiment', title_fontsize=11, fontsize=10)
ax.set_xticklabels(pivot_entity_negative.index, rotation=45, ha='right')

# Add grid for better readability
ax.yaxis.grid(True, alpha=0.3)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()

print("\nNote: Entities are ordered left-to-right by negative mention count (highest first).")
```




```
Note: Entities are ordered left-to-right by negative mention count (highest first).
```

Observation

This graph brings it all together; on the farthest left side should be the entities with the highest (if not 100%) negative seniment, decreasing to the left, where there are entities with more neutral and/or positive sentiment to the right. A client might use this graph to NOT ONLY determine out of all entities, which ones to avoid the most, but also what entities have the highest negative sentiment, while still having the *least bit* of positive (or neutral) sentiment.

This prioritization helps e-commerce clients quickly identify high-risk entities while understanding their full sentiment context. An entity with 100 negative mentions and 200 positive mentions tells a different story than one with 100 negative mentions and 0 positive mentions - both are shown here, but ordered by the negative signal strength.

Analyzing Confidence Scores for Sentiment

These visualizations will explore the confidence scores of entites in the negative and positive classifications. WiggINet utilizes the sentiment analysis model (RoBERTa) which provides confidence scores indicating how certain it is about the sentiment classification. Examining confidence alongside negative mentions helps distinguish between high and low congfidence entities. High confidence entities represent reliable warning signals, and low confidence entities can be potentially misclassified or ambiguous data.

Define Reusable Visualization Functions

Taking a new approach starting this vizualization, all repeating graphing data will be stored in a function to avoid code duplication.

Data Preparation

The function `plot_confidence_bar_chart` creates a horizontal bar chart with color gradient based confidence. It takes four arguments: the data, mention counts, confidence scores, sentiment type and colormap for each of the two graphs that will be generated for each top positive and negative entities.

In [53]:
```
import matplotlib.cm as cm
from matplotlib.colors import Normalize

def plot_confidence_bar_chart(data, mentions_col, confidence_col,
                              sentiment_type, color_map='Reds'):
    # sort by mentions for better visualization
    data_sorted = data.sort_values(mentions_col, ascending=True)

    # create figure object and ax
    fig, ax = plt.subplots(figsize=(12, 10))

    # normalize the confidence scores across the data,
    #   so that it can be color mapped
    norm = Normalize(vmin=data_sorted[confidence_col].min(),
                     vmax=data_sorted[confidence_col].max())
    cmap = cm.get_cmap(color_map) # call in color map

    # Create bars with color based on confidence
    ax.barh(range(len(data_sorted)),
            data_sorted[mentions_col],
            color=cmap(norm(data_sorted[confidence_col])))

    # Customize chart
    ax.set_yticks(range(len(data_sorted)))
    ax.set_yticklabels(data_sorted['ENTITY_TEXT'], fontsize=10)
    ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
    ax.set_ylabel('Entity', fontsize=12, fontweight='bold')
    ax.set_title(f'Top 25 {sentiment_type}ly Mentioned Entities\n(Color Intensity = Confidence Level)',
                 fontsize=14, fontweight='bold', pad=20)

    # Add grid
    ax.xaxis.grid(True, alpha=0.3)
    ax.set_axisbelow(True)

    # Add colorbar
    sm = cm.ScalarMappable(cmap=cmap, norm=norm)
    sm.set_array([])
    cbar = plt.colorbar(sm, ax=ax)
    cbar.set_label('Average Confidence Score', fontsize=11, fontweight='bold')

    # Add value labels on bars
    for i, (mentions, confidence) in enumerate(zip(data_sorted[mentions_col],
                                                   data_sorted[confidence_col])):
        ax.text(mentions + 1, i, f'{int(mentions)} ({confidence:.2f})',
                va='center', fontsize=8, color='black')

    plt.tight_layout()
    plt.show()
```

This function, `plot_confidence_scatter` creates a scatter plot of confidence compared with mentions with quadrant analysis. The arguments are mostly the same as the bar graph, but high and low quadrant args are added and max_mentions for a total.

In [54]:
```
def plot_confidence_scatter(data, mentions_col, confidence_col,
                            sentiment_type, color_map='Reds',
                            quadrant_color_high='#ffcccc', quadrant_color_low='#ffffcc',
                            max_mentions=None):
    # Create figure
    fig, ax = plt.subplots(figsize=(14, 8))

    # Plot scatter points
    scatter = ax.scatter(data[mentions_col],
                         data[confidence_col],
                         s=200,
                         c=data[confidence_col],
                         cmap=color_map,
                         alpha=0.6,
                         edgecolors='black',
                         linewidth=1.5)

    # Add entity labels
    for idx, row in data.iterrows():
        ax.annotate(row['ENTITY_TEXT'],
                    (row[mentions_col], row[confidence_col]),
                    fontsize=9,
                    xytext=(5, 5),
                    textcoords='offset points',
                    bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow', alpha=0.3))

    # Add quadrant lines (median splits)
    median_mentions = data[mentions_col].median()
    median_confidence = data[confidence_col].median()

    ax.axvline(median_mentions, color='gray', linestyle='--', alpha=0.5, linewidth=1)
    ax.axhline(median_confidence, color='gray', linestyle='--', alpha=0.5, linewidth=1)

    # Add quadrant labels
    ax.text(median_mentions * 1.5, data[confidence_col].max() * 0.98,
            'High Confidence\nHigh Volume', ha='center', fontsize=10,
            bbox=dict(boxstyle='round', facecolor=quadrant_color_high, alpha=0.7))
    ax.text(median_mentions * 0.5, data[confidence_col].max() * 0.98,
            'High Confidence\nLow Volume', ha='center', fontsize=10,
            bbox=dict(boxstyle='round', facecolor=quadrant_color_low, alpha=0.7))

    # Customize chart
    ax.set_xlabel(f'Total {sentiment_type} Mentions', fontsize=12, fontweight='bold')
    ax.set_ylabel('Average Confidence Score', fontsize=12, fontweight='bold')
    ax.set_title(f'Confidence vs. {sentiment_type} Mention Count for Top 25 Entities',
                 fontsize=14, fontweight='bold', pad=20)
    ax.grid(True, alpha=0.3)

    # Cap the x-axis if max_mentions is specified
    if max_mentions is not None:
        ax.set_xlim(0, max_mentions)

    # Add colorbar
    cbar = plt.colorbar(scatter, ax=ax)
    cbar.set_label('Confidence Score', fontsize=11, fontweight='bold')

    plt.tight_layout()
    plt.show()
```

Data Preparation

Filter the existing dataset for negative sentiment and aggregate by entity to calculate total negative mentions and verage confidence score across all negative mentions.

In [55]:
```
# Filter the existing dataframe for negative and positive sentiment
df_negative = df[df['SENTIMENT'] == 'negative'].copy()
df_positive = df[df['SENTIMENT'] == 'positive'].copy()

print(f"Found {len(df_negative)} negative sentiment records")
print(f"Confidence score range: {df_negative['CONFIDENCE'].min():.3f} to {df_negative['CONFIDENCE'].max():.3f}")
print(f"Average confidence: {df_negative['CONFIDENCE'].mean():.3f}\n")

print(f"Found {len(df_positive)} positive sentiment records")
print(f"Confidence score range: {df_positive['CONFIDENCE'].min():.3f} to {df_positive['CONFIDENCE'].max():.3f}")
print(f"Average confidence: {df_positive['CONFIDENCE'].mean():.3f}\n")
```

```
Found 255 negative sentiment records
Confidence score range: 0.519 to 0.825
Average confidence: 0.674

Found 7222 positive sentiment records
Confidence score range: 0.483 to 0.978
Average confidence: 0.768
```

Sum the mentions and find the mean confidence to put ina variable called `negative_entity_stats` .

In [56]:
```
# Aggregate NEGATIVE entities: sum mentions and average confidence
negative_entity_stats = df_negative.groupby('ENTITY_TEXT').agg({
    'MENTIONS': 'sum',
    'CONFIDENCE': 'mean'
}).reset_index()

negative_entity_stats.columns = ['ENTITY_TEXT', 'TOTAL_NEGATIVE_MENTIONS', 'AVG_CONFIDENCE']
top_25_negative = negative_entity_stats.nlargest(25, 'TOTAL_NEGATIVE_MENTIONS')
top_25_negative['AVG_CONFIDENCE'] = top_25_negative['AVG_CONFIDENCE'].round(3)

# Aggregate POSITIVE entities: sum mentions and average confidence
positive_entity_stats = df_positive.groupby('ENTITY_TEXT').agg({
    'MENTIONS': 'sum',
    'CONFIDENCE': 'mean'
}).reset_index()

positive_entity_stats.columns = ['ENTITY_TEXT', 'TOTAL_POSITIVE_MENTIONS', 'AVG_CONFIDENCE']
top_25_positive = positive_entity_stats.nlargest(25, 'TOTAL_POSITIVE_MENTIONS')
top_25_positive['AVG_CONFIDENCE'] = top_25_positive['AVG_CONFIDENCE'].round(3)
```

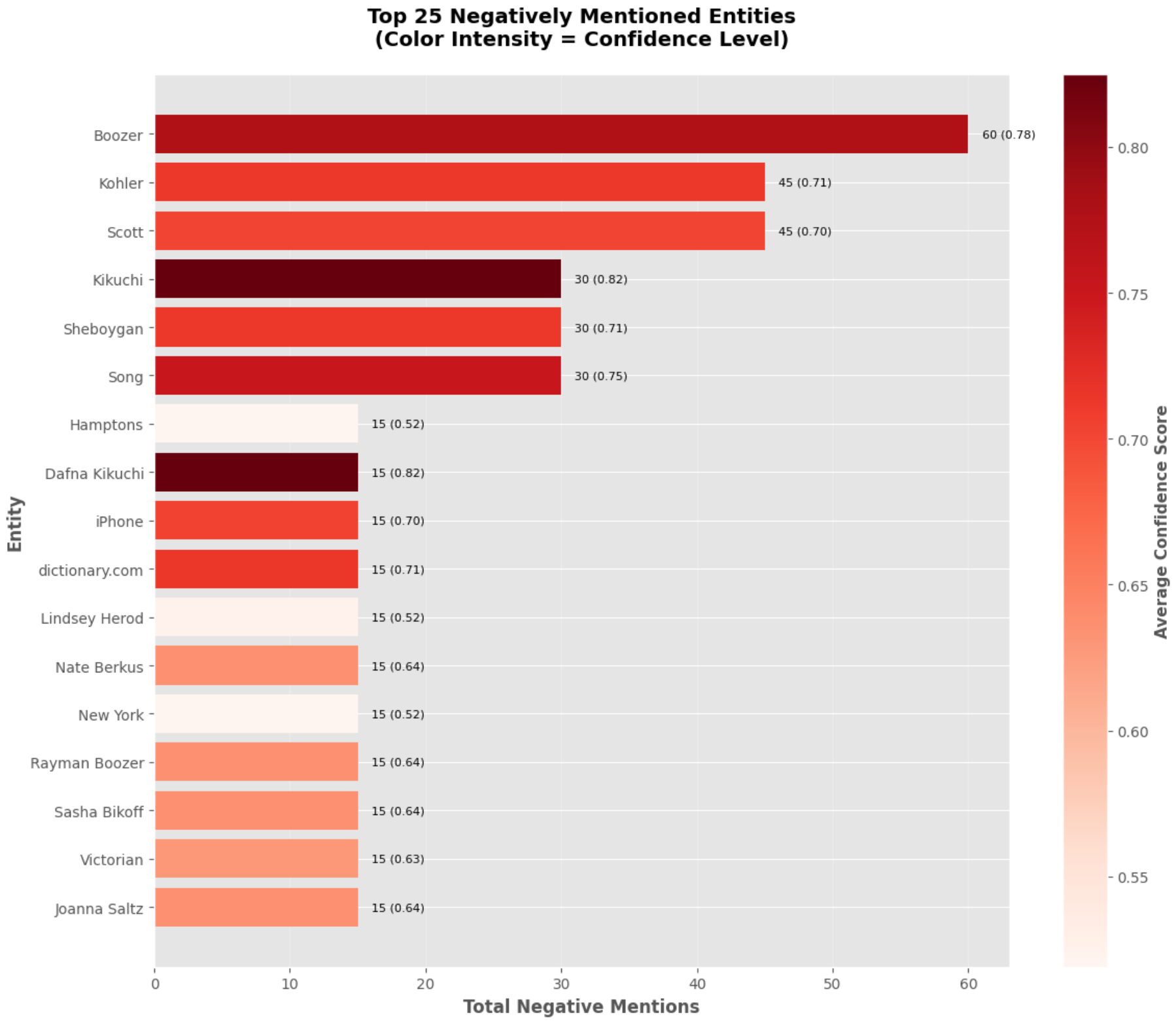
Visualization

Bar Chart with Color Gradient by Confidence

Bar height represents negative mention count, while color intensity shows confidence level. Darker bars indicate higher model confidence in the negative sentiment classification.

In [57]:
```
# Use the reusable function for negative entities bar chart
plot_confidence_bar_chart(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds'
)
```

```
/tmp/ipykernel_8994/2958685450.py:16: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplo
tlib 3.7 and will be removed two minor releases later. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormap
s.get_cmap(obj)`` instead.
  cmap = cm.get_cmap(color_map) # call in color map
```

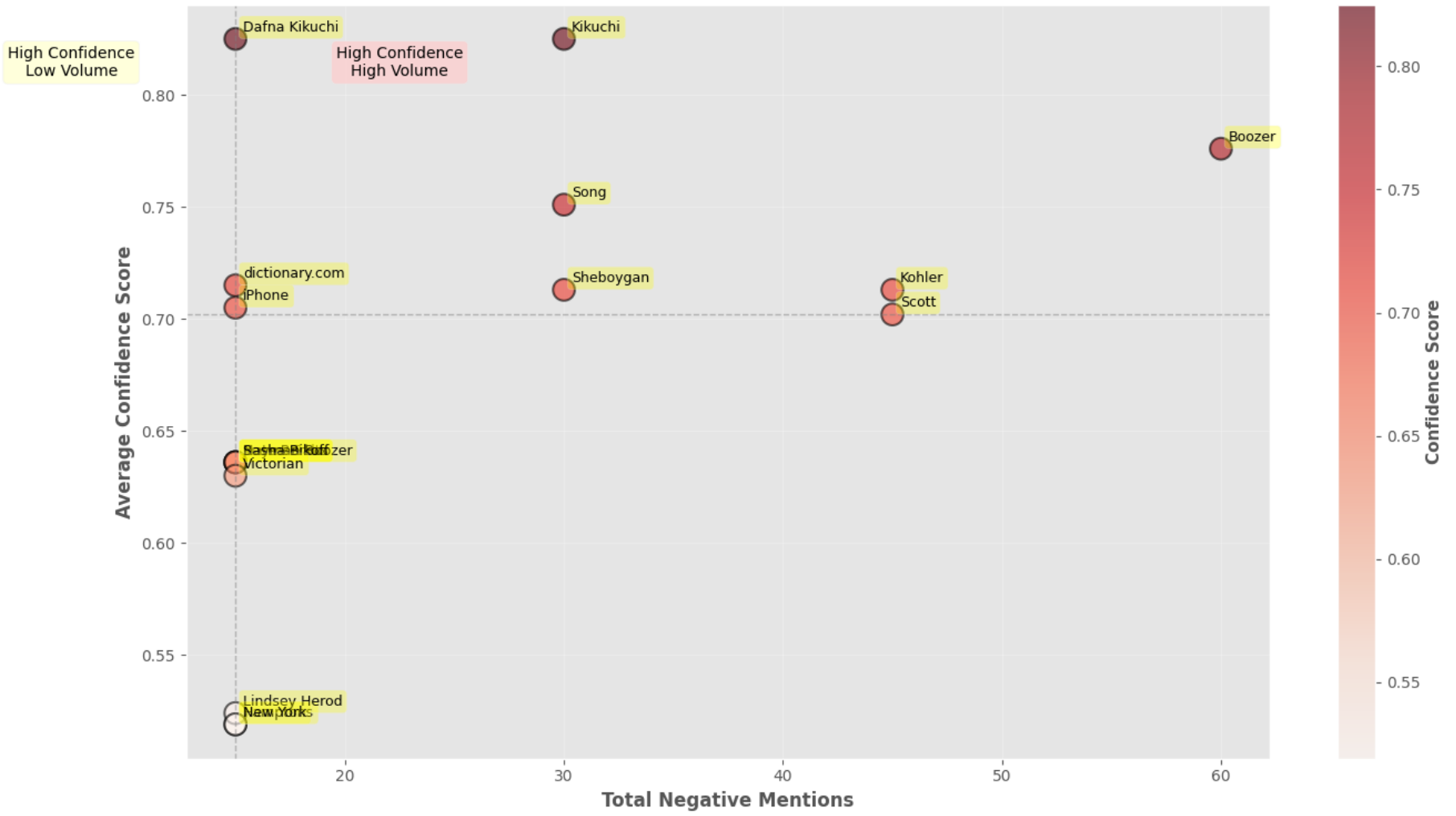


Scatter Plot - Confidence vs. Negative Mentions

In the below scatter plot, the top right quadrant represent high mentions and high confidence which symbolize the most reliable warning signs.

To the left, low mentions with high confidence indicate emerging concerns. The bottom right quadrant with high mentions and low confidence could be potentially misclassified data points. At the bottom left-hand side, low mentions and low confidence entities pose the least concern.

In [58]:
```
# Use the reusable function for negative entities scatter plot
plot_confidence_scatter(
    data=top_25_negative,
    mentions_col='TOTAL_NEGATIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Negative',
    color_map='Reds',
    quadrant_color_high='#ffcccc',
    quadrant_color_low='#ffffcc'
)
```



As mentioned before, entities on the farthest right side of the scatter plot have higher mentions, but those in the upper porton of the right side have high confidence scores.

These are what the difference confidence thresholds mean.

(>0.085) - the RoBERTa model is very clean about the sentiment classification
(0.70-0.85) - the model is moderately certain, which is typical for clear negative language
(<0.70) - low confidence: uncertain classification. May indicate ambiguous language or misclassification

For WigglNet clients, high confidence, high volume entities will show up as darker, taller bars whic indicate the most reliable negative signals. E-commerce sellers should investigate these entities thoroughly before associating with them.

This two-dimensional analysis helps clients distinguish between solid negative signals and potentially uncertain classifications, leading to better-informed business decisions.

Positive Sentiment Confidence Analysis

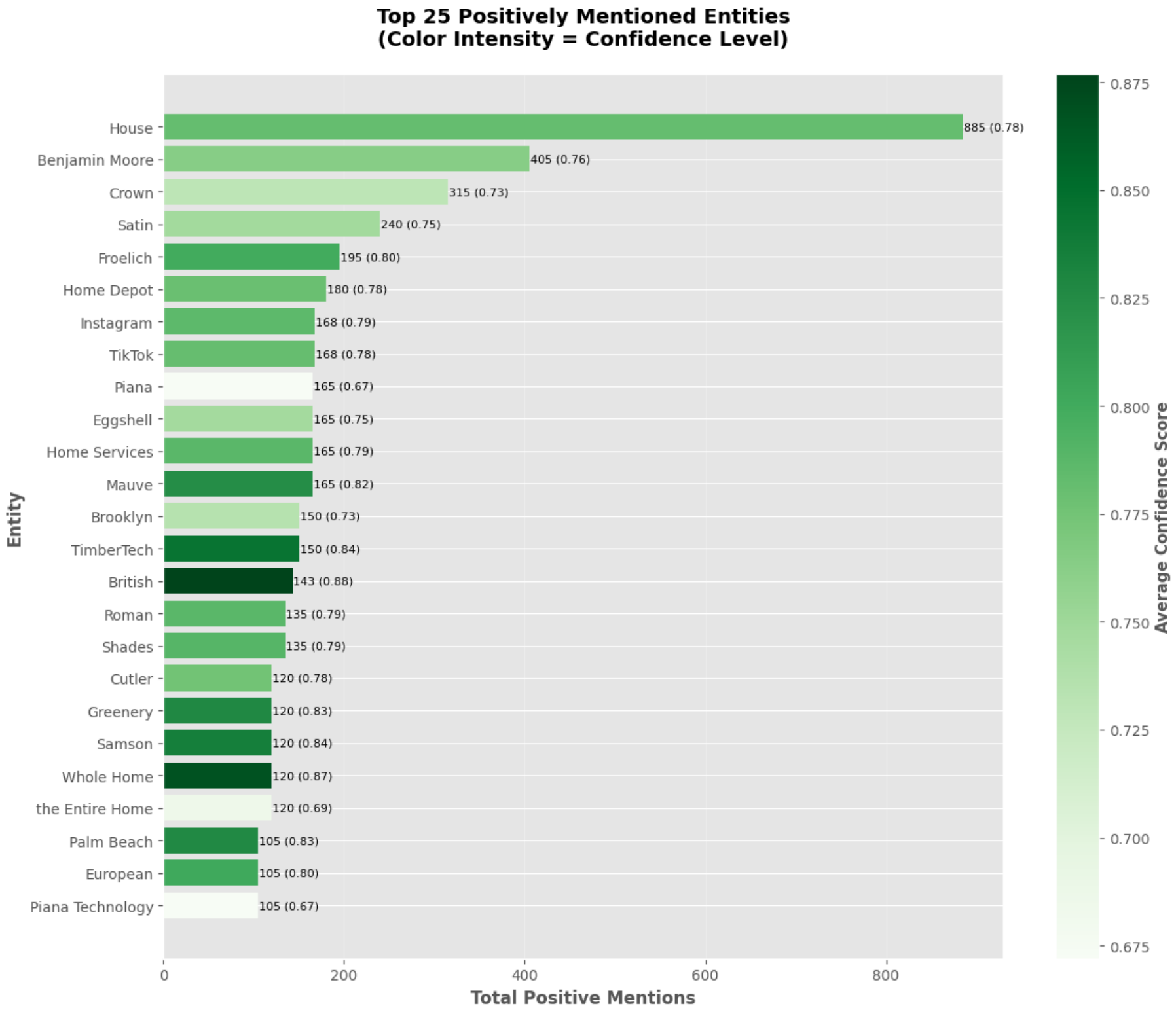
Now analyzing the top 25 positively mentioned entities with their confidence scores to identify reliable positive signals.

Bar Chart with Color Gradient by Confidence (Positive Entities)

Bar height represents positive mention count, while color intensity shows confidence level. Darker bars indicate higher model confidence in the positive sentiment classification.

In [59]:
```
# Use the reusable function for positive entities bar chart
plot_confidence_bar_chart(
    data=top_25_positive,
    mentions_col='TOTAL_POSITIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Positive',
    color_map='Greens'
)
```

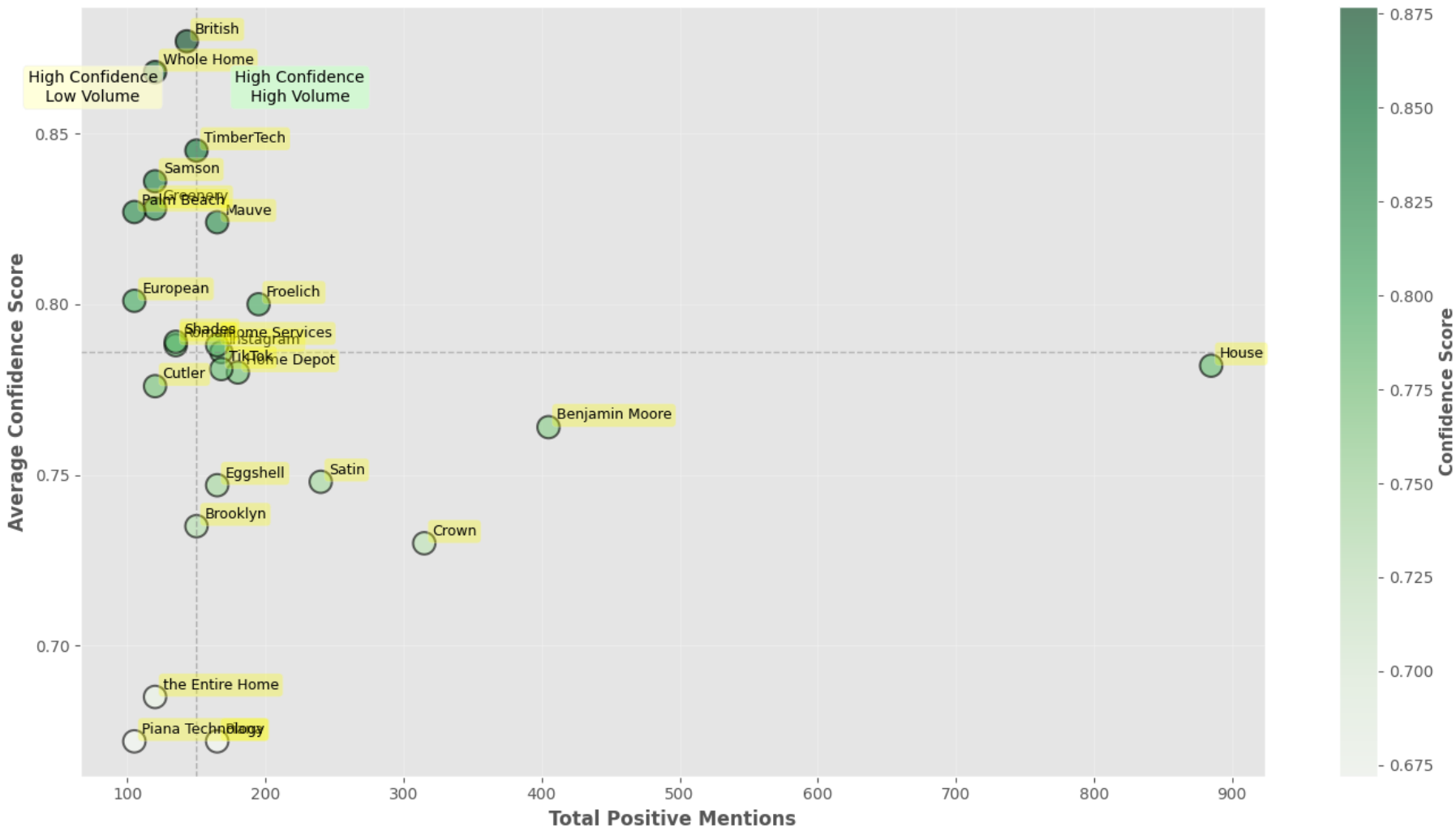
```
/tmp/ipykernel_8994/2958685450.py:16: MatplotlibDeprecationWarning: The get_cmap function was deprecated in Matplo
tlib 3.7 and will be removed two minor releases later. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormap
s.get_cmap(obj)`` instead.
  cmap = cm.get_cmap(color_map) # call in color map
```



Similar to the negative sentiment analsysis, this plot helps identify patterns in positive sentiment. All the same quadrant definitions still apply, except the entities with postitve sentiment and the most mentions should tell client how safe certain entities are.

Low mentions with high confidece represent emerging positive while low mentions with high confidence indicate entities that are least significant.

In [60]:
```
# Use the reusable function for positive entities scatter plot
plot_confidence_scatter(
    data=top_25_positive,
    mentions_col='TOTAL_POSITIVE_MENTIONS',
    confidence_col='AVG_CONFIDENCE',
    sentiment_type='Positive',
    color_map='Greens',
    quadrant_color_high='#ccffcc',
    quadrant_color_low='#ffffcc'
)
```



Observation

This positive sentiment confidence chart shows a similar representation to the negative sentiment confidence chart. It can be read the same way, except the meaning is obviously the opposite. E-commerce seller should consider partnering with or promoting entities that fall within the high-confidence, high-volume areas. These could make strong candidates for marketing and product development.

Meanwhile, low confidence positive entities could indicate mixed messaging.

This is how a client might use this graph:

Positive Confidence + Volume = Opportunity Score

- **High mentions × High confidence = Prime opportunity (promote/partner)**
- **High mentions × Low confidence = Needs review (investigate further)**
- **Low mentions × High confidence = Emerging trend (monitor closely)**
- **Low mentions × Low confidence = Low priority**

Comparing Positive vs. Negative: By analyzing both positive and negative entities with confidence scores, WiggINet clients can make balanced decisions:

- **Entities appearing in both lists may have polarizing reputations**
- **Pure positive entities (high confidence) are safe bets for partnerships**
- **Entities with high positive mentions but low confidence may need deeper investigation before committing**

Conclusion

The best way for a seller or manufacturer to understand what's happening in a market is to look directly at what is being talked about. Reading articles one by one and making judgements for yourself whether an entity is positive or negative is an inefficient approach compared to the classification models WiggINet runs to identify the sentiment of keywords and entities accross hundreds of articles.

This notebook identified `sentiment` as the target feature for predicting the vibe of keywords and entities in a particular product category. It was visualized in three stacked bar charts showing the different ways clients could use this data to make decisions about certain topics between positive and negative sentiment.